

RNN의 Recurrent Nerual Net의 줄임말(2)

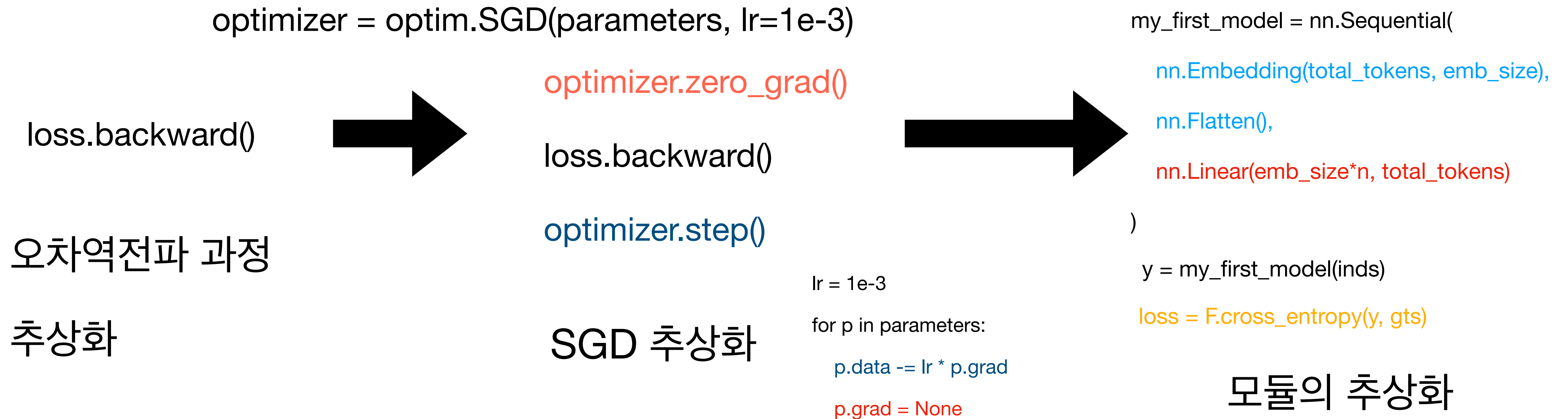
todo: 멋진 것 만들기

뉴비..를 위한 인공지능이 재밌어지는 시간! - REVIEW

1. output을 next token 확률분포로
2. embedding을 사용하자!
3. pytorch에서 모델을 정의하고 학습시키는 방법
4. 비선형성이 중요한 이유
5. RNN은 Recurrent Neural Net의 줄임말

뉴비..를 위한 인공지능이 재밌어지는 시간! - REVIEW

지난 시간에 numpy 손코딩을 pytorch로 바꾸어보자...



뉴비...를 위한 인공지능이 재밌어지는 시간! - 비선형성이 필요해요11

Universal approximation Theorem

compact K 위에서 연속 함수 $f(x)$

there exists $k \in \mathbb{N}, A \in \mathbb{R}^{k \times n}, b \in \mathbb{R}^k, C \in \mathbb{R}^{m \times k}$

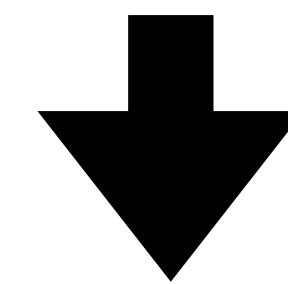
s.t. $\sup_{x \in K} |f(x) - g(x)| < \epsilon$

where, $g(x) = C(\sigma(Ax + b))$

흥미로운 지점

은닉층이 하나여도 $\neg$ 첫

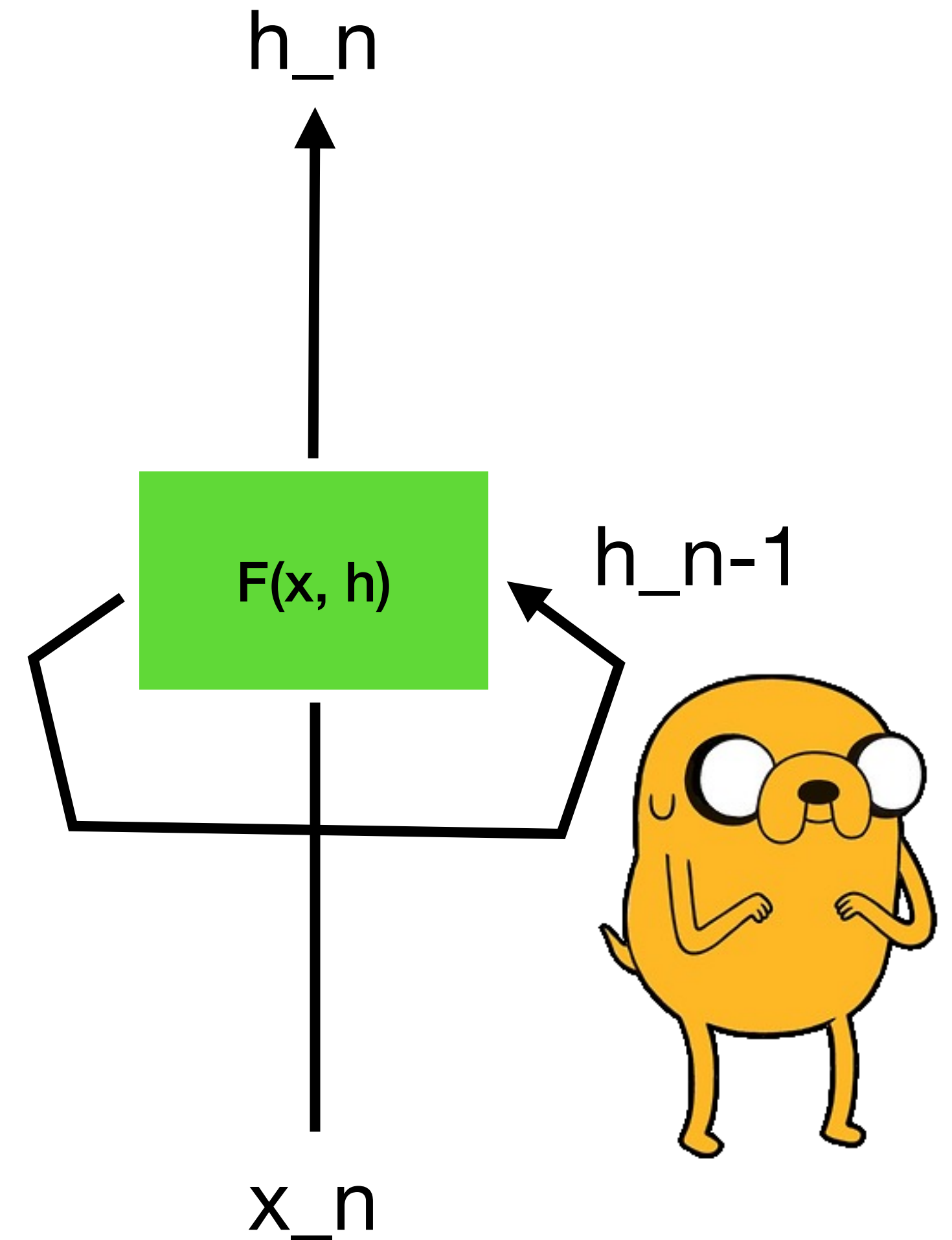
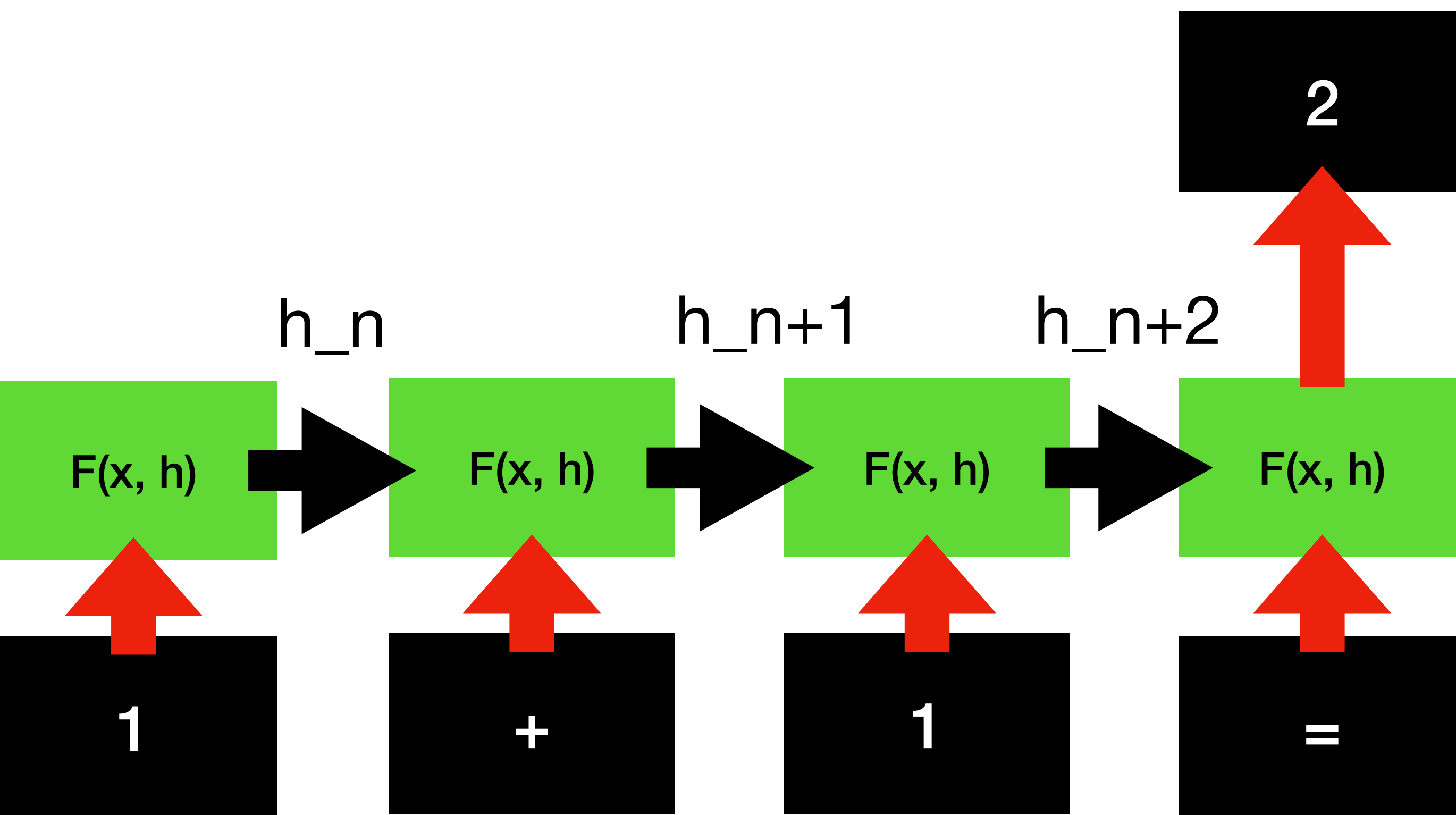
효율적인 것은 아님



Engineering이 필요한 이유

https://en.wikipedia.org/wiki/Universal_approximation_theorem

find $F(x)$? - rnn



<https://bit.ly/4peZVrl>

Attention 마예요?

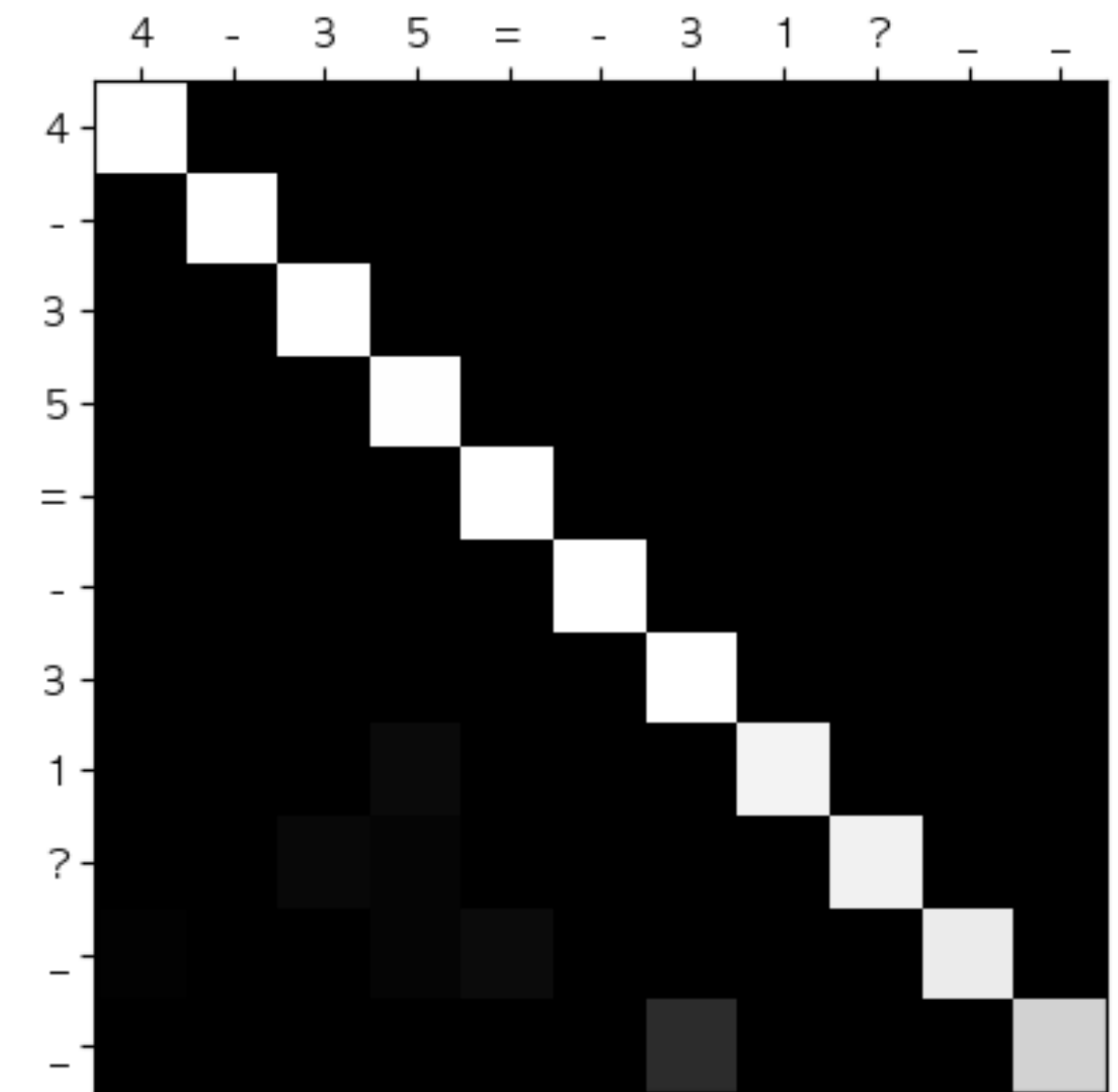


Attention

“어디에” 집중하고 싶어요.

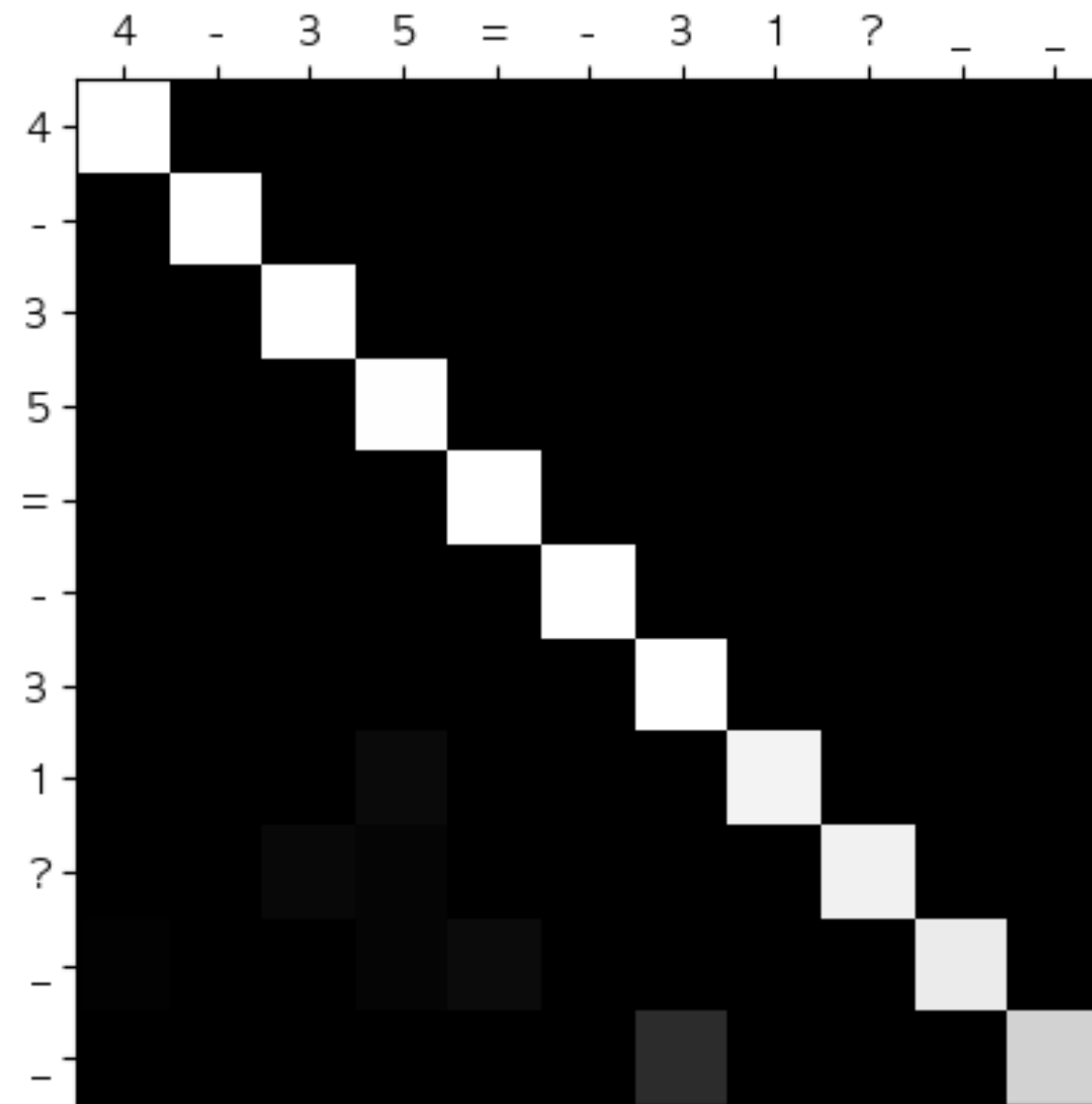
그 어디를 구하는 방법 = cosine similarity

정보를 재조합한다!



Attention 머예요?

“어디에” 집중하고 싶어요.



```
attention_weight = F.softmax((proj @ torch.transpose(proj, 1, 2)) / math.sqrt(emb_size) * mask, dim=-1)
outs = attention_weight @ proj
return outs
```

mask가 필요합니다!

1 + 1 = 23?



맞춰야 하는 것

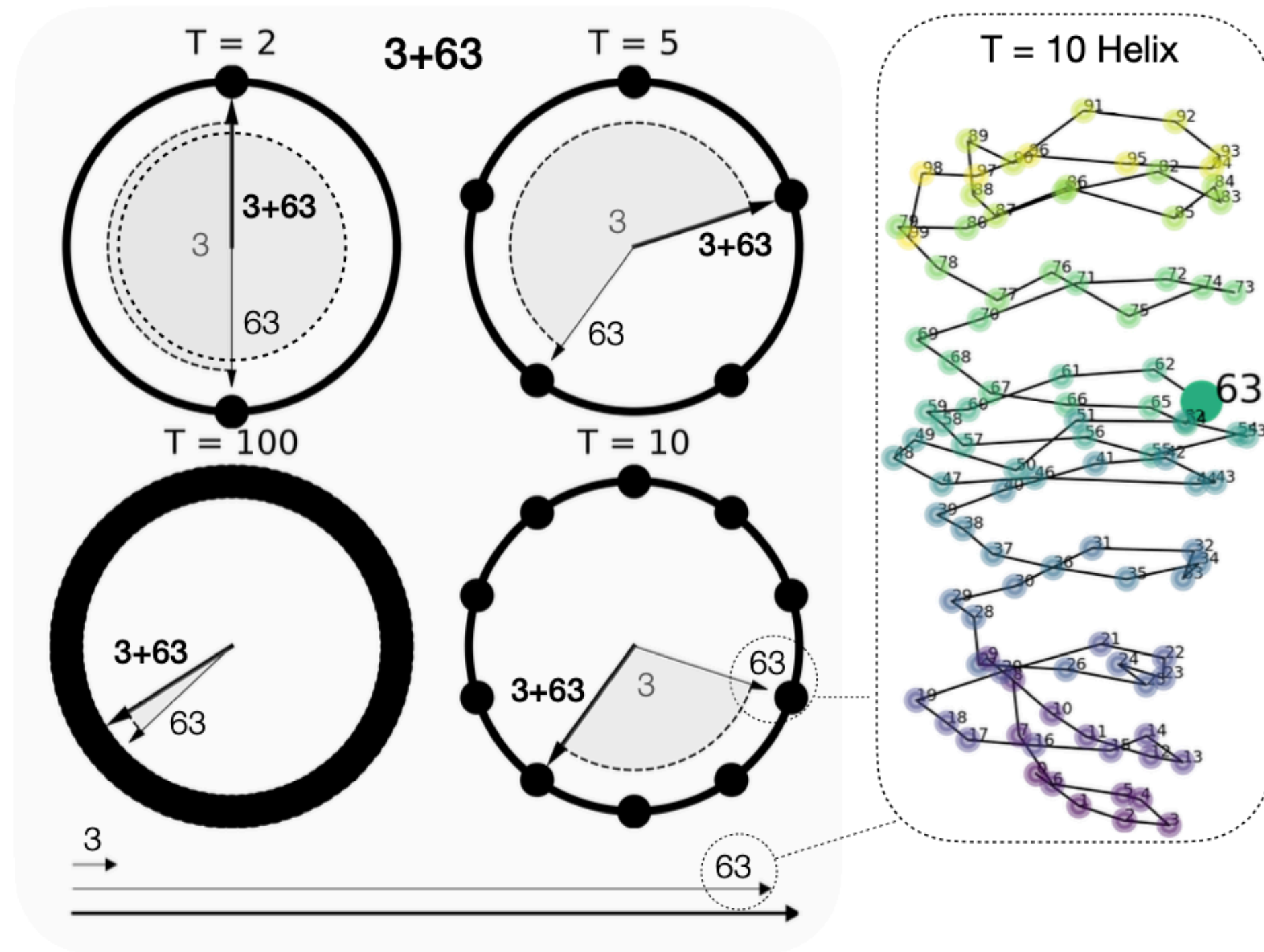
LLM은 어떻게 더해요?

$$a + b =$$

Step 1: Embed a and b as a helix using modular circles ($T = [2, 5, 10, 100]$) and a line

Step 2: Create $\text{helix}(a + b)$

Step 3: Translate answer helix to logits



2. A sparse set of attention heads, mostly in layers 9-14, move the a and b helices to the last token.
3. MLPs 14-18 manipulate these helices to create the helix $a + b$. A small number of attention heads help with this operation.
4. MLPs 19-27 and a few attention heads “read” from the $a + b$ helix and output to model logits.

삼각함수 덧셈 정리를 떠올려 보면...

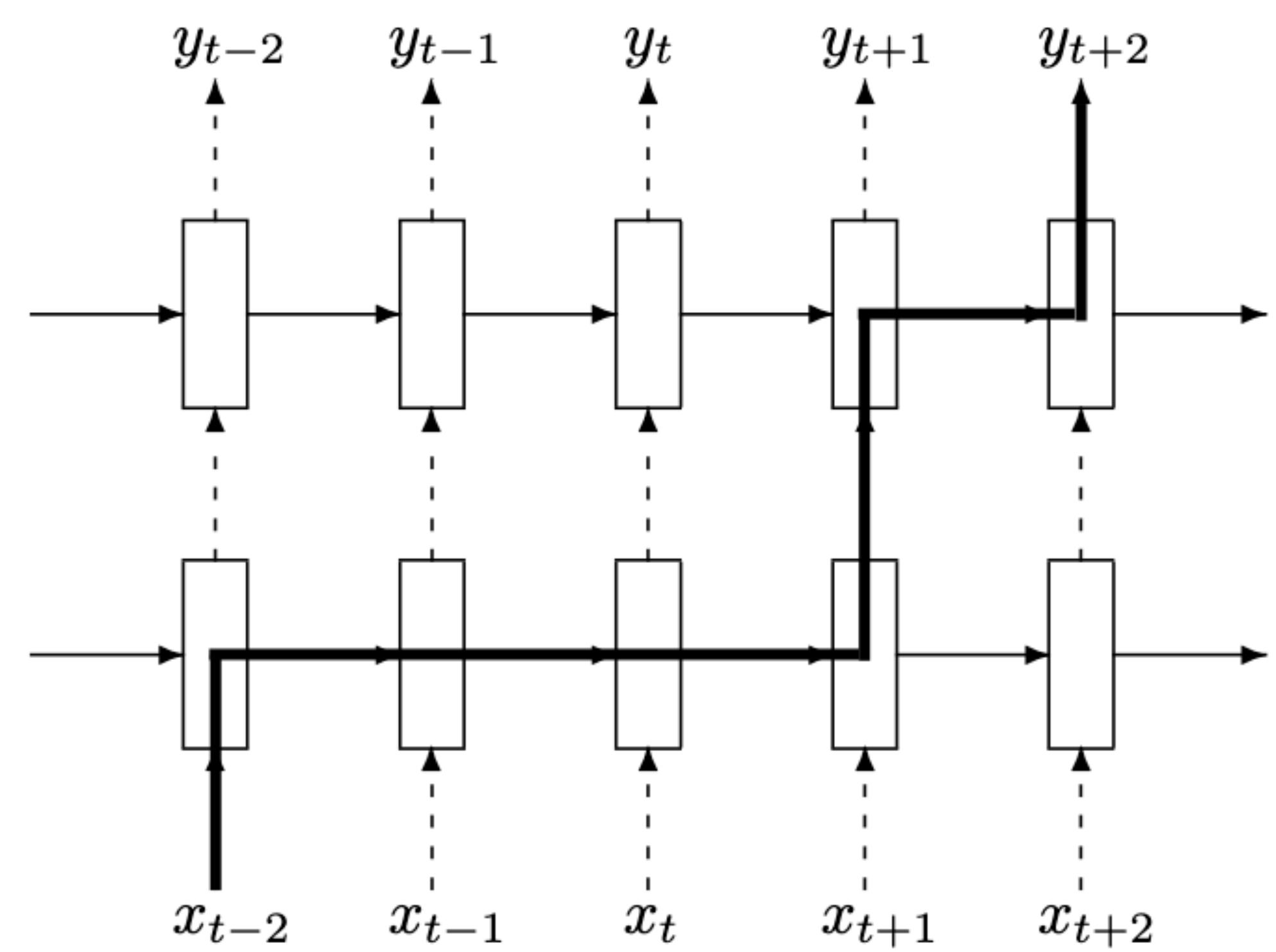
$$\cos(t+w) = \cos(t)\cos(w) - \sin(t)\sin(w)$$

아니 $\sin(t)$, $\sin(w)$ 는 어케 구함?

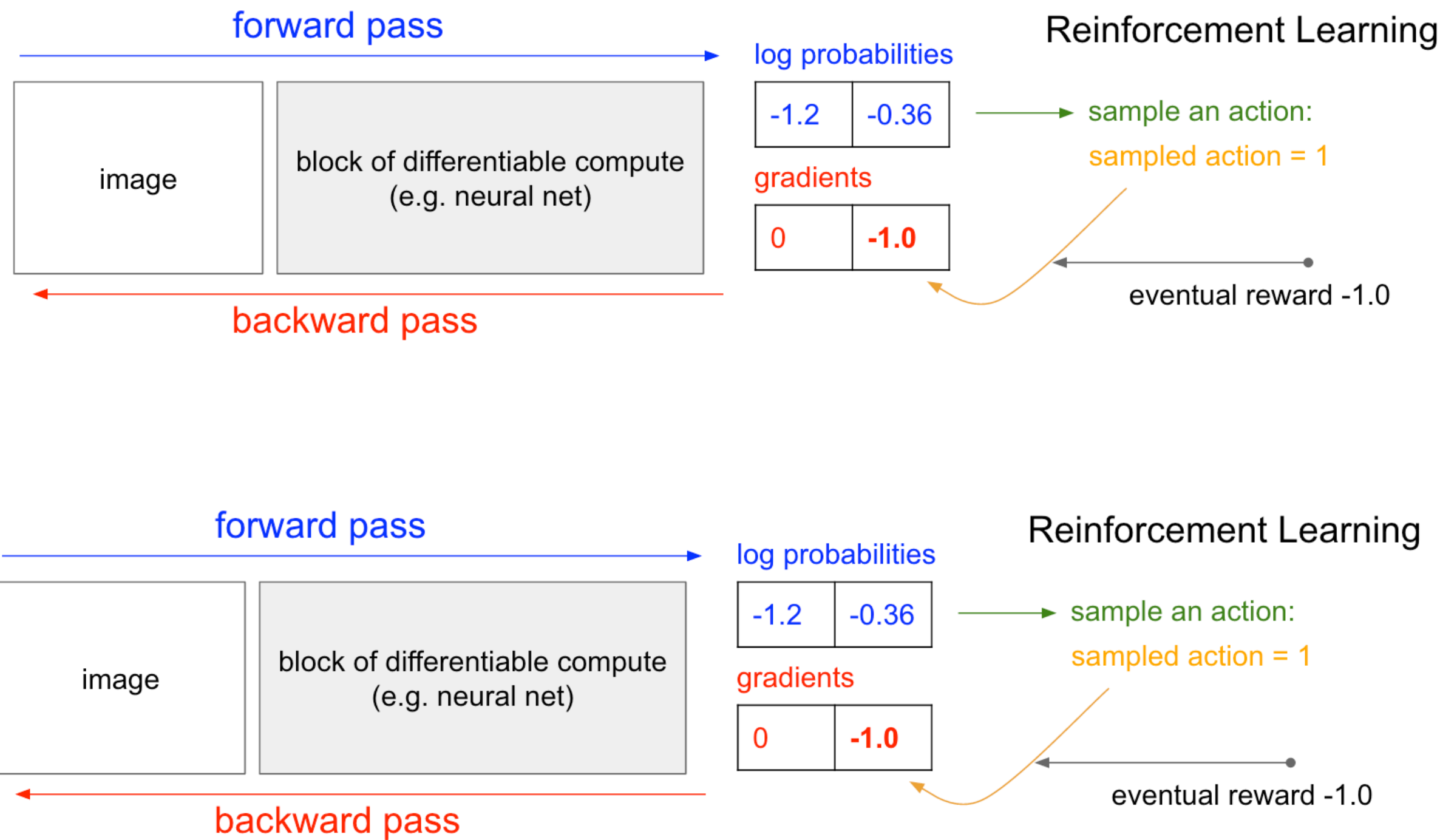
taylor expansion! -> 1차항까지..

MLP로 가능..! = 배울 가능성이 있다.

조금 더 넣어볼거예요



Dropout...



강화학습은...다음에